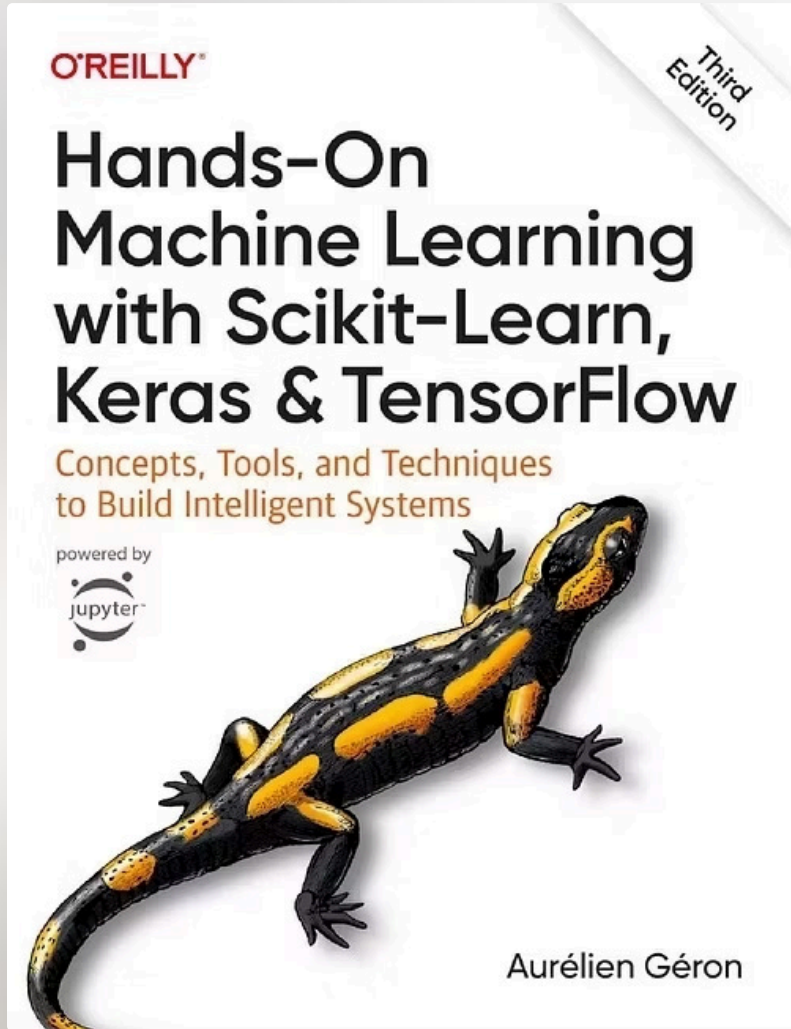




Machine Learning

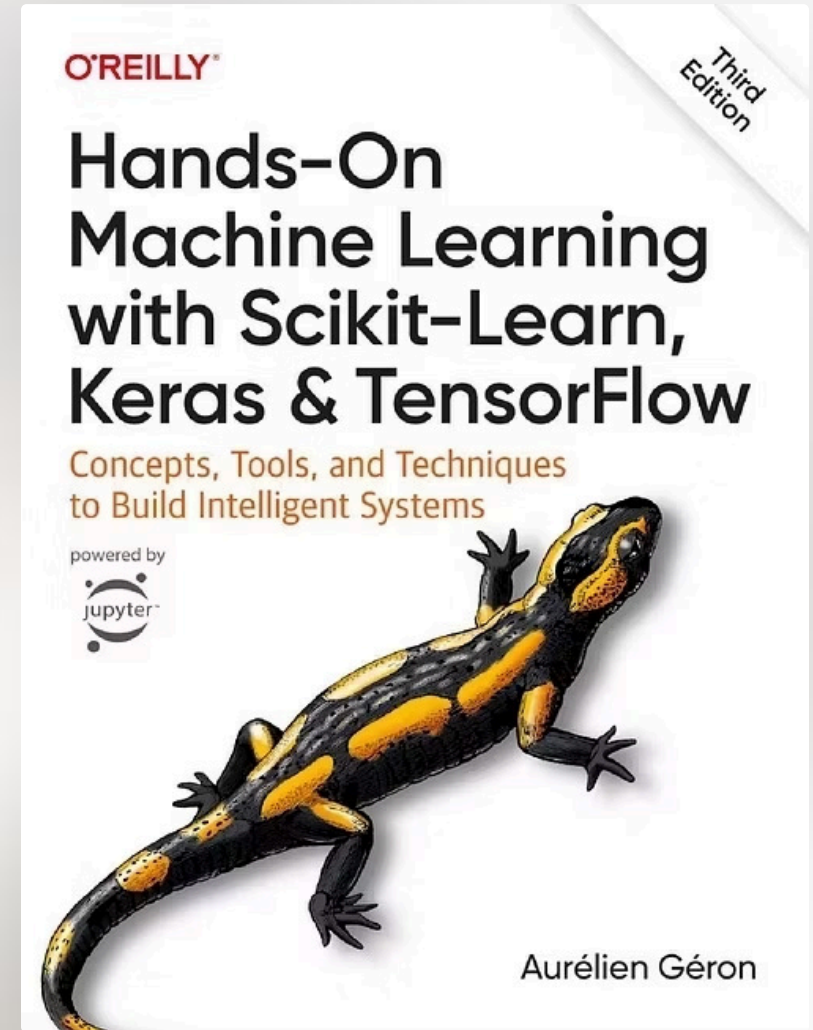
Lessons based on the Textbook "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron, 2023.

RP by Rodrigo Martins Pagliares

Last update: 07/11/2025

Machine Learning

Chapter 1 – The Machine Learning Landscape





Agenda for Chapter 1

1. What Is Machine Learning?
2. Why Use Machine Learning?
3. Examples of Applications
4. **Types of Machine Learning Systems**
5. Main Challenges of Machine Learning
6. Testing and Validating

Agenda for Chapter 1

1. What Is Machine Learning?
2. Why Use Machine Learning?
3. Examples of Applications
4. **Types of Machine Learning Systems**
 - Classification criteria #1: Supervision during training
 - Supervised, Unsupervised, Semi-Supervised, Self-Supervised, and Reinforcement Learning
 - Classification criteria #2: Approach to Learning Method
 - Batch and Online (Incremental) Learning
 - **Classification criteria #3: Learning Method**
 - Instance-Based and **Model-Based Learning**
5. **Main Challenges of Machine Learning**
 - Insufficient Quantity of Training Data
 - Non-representative Training Data
 - Poor-Quality Data
 - Irrelevant Features
 - Overfitting the Training Data
 - Underfitting the Training Data
 - Stepping Back
6. **Testing and Validating**
 - Hyperparameter Tuning and Model Selection
 - Data Mismatch



Classification criteria #3: Learning Method

Instance-Based and Model-Based Learning

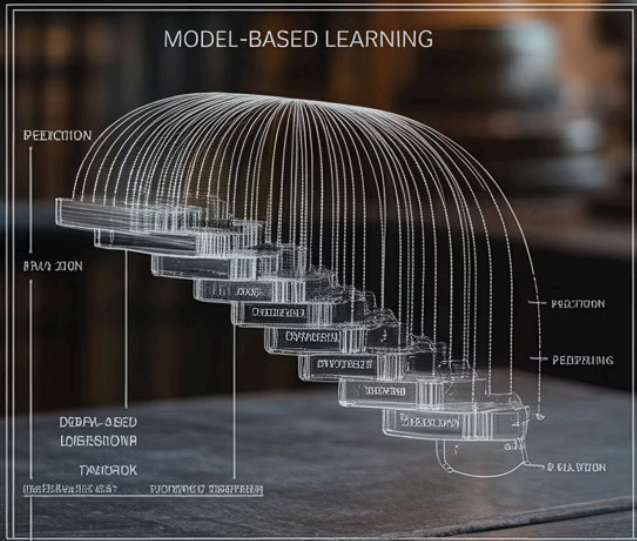
Model-Based Learning: Overview

- **Definition:**
 - Instead of memorizing examples, the system **builds a model** from the **training data**.
 - The **model** is then used to make **predictions** on **new, unseen data**.



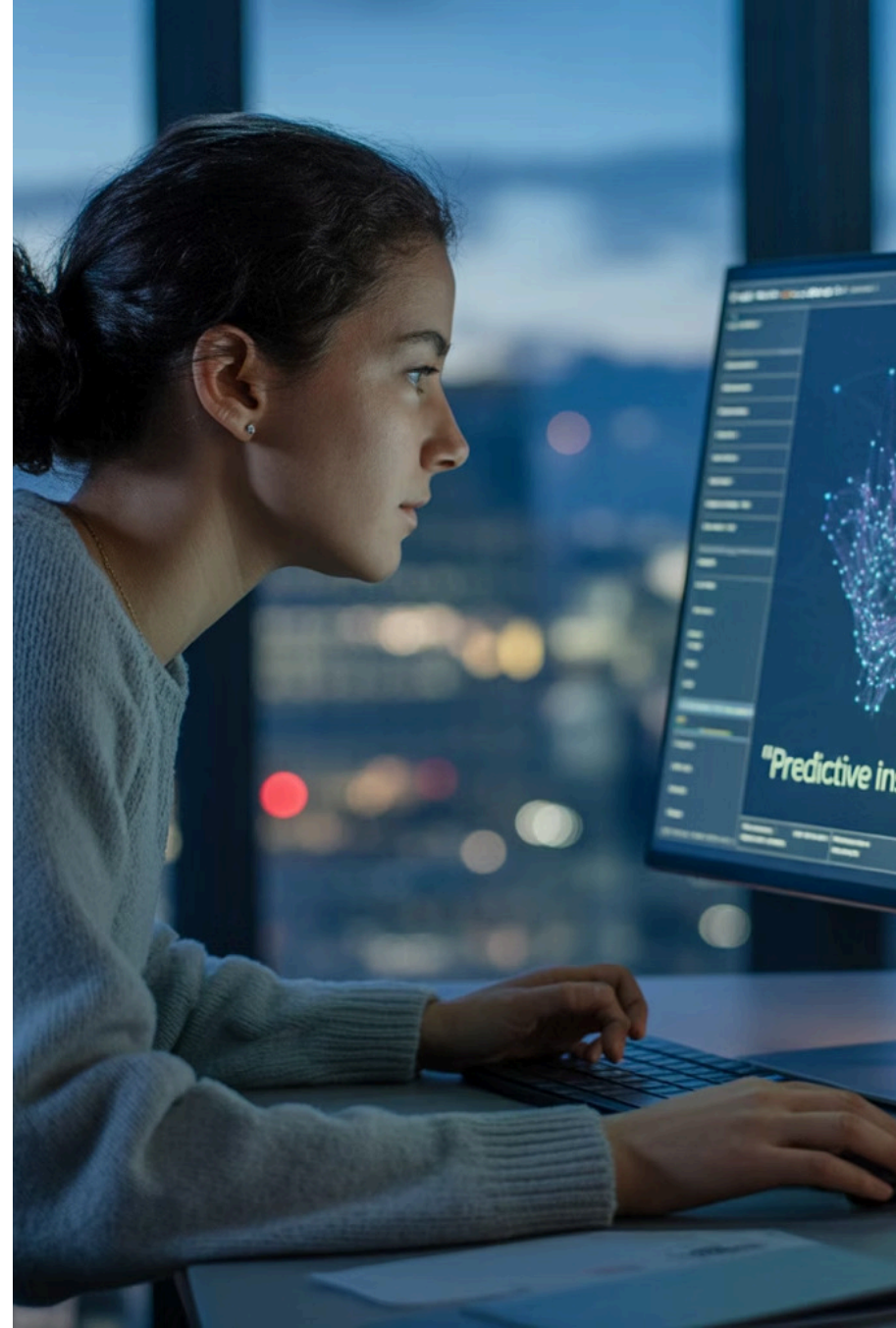
How Model-Based Learning Works

- **Process:**
 - Builds a predictive model that captures **patterns** and **relationships** in the **data**.
 - Once the model is **trained**, it makes **predictions** by applying the **learned patterns**.
- **Example:**
 - A **decision tree** or a **neural network** can model **relationships** between **features** and **target variables**.



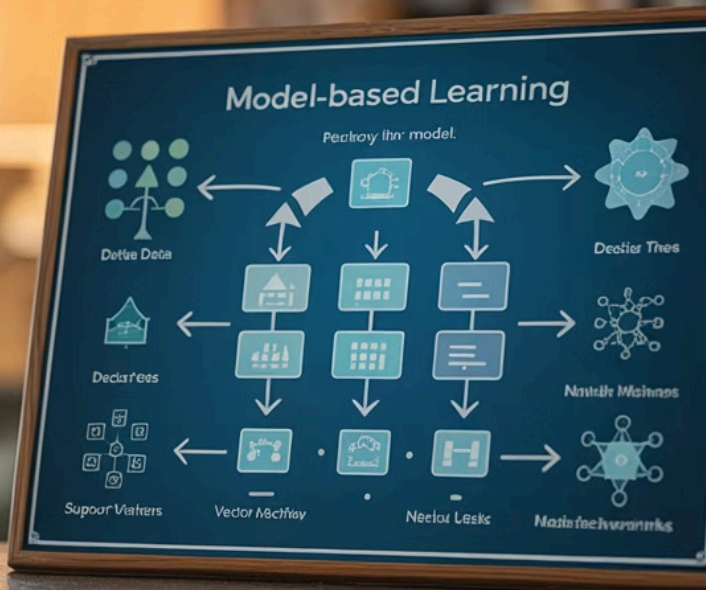
Example: Model-Based Learning in Action

- Task:
 - Given a **set of training data**, the **model identifies** underlying **patterns** or **rules**.
 - **Prediction**: Once trained, the **model predicts** new **instances** based on the **patterns** it has **learned**.



Key Takeaways

- **Model-Based Learning:**
 - Involves building a **model** from the **data**, which is used to make **predictions** on **new data**.
- **Advantages:**
 - Typically more efficient for large datasets than **instance-based learning**.
 - **Examples:** Decision trees, support vector machines (SVM), neural networks.



Model-Based Learning

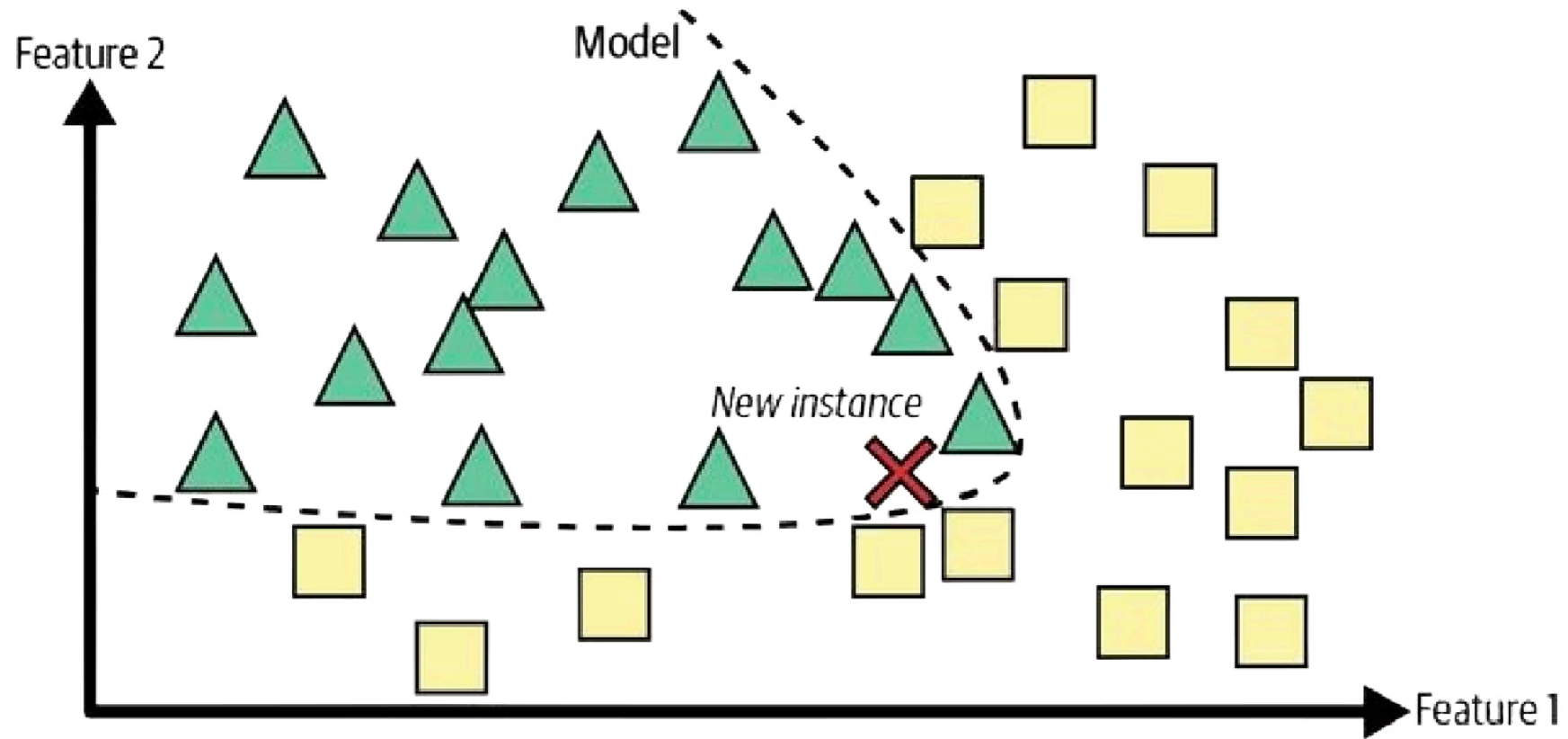


Figure 1-17. Model-based learning



The next set of slides is about an example that analyses the Relationship Between Money and Happiness

Example: Analyzing the Relationship Between Money and Happiness

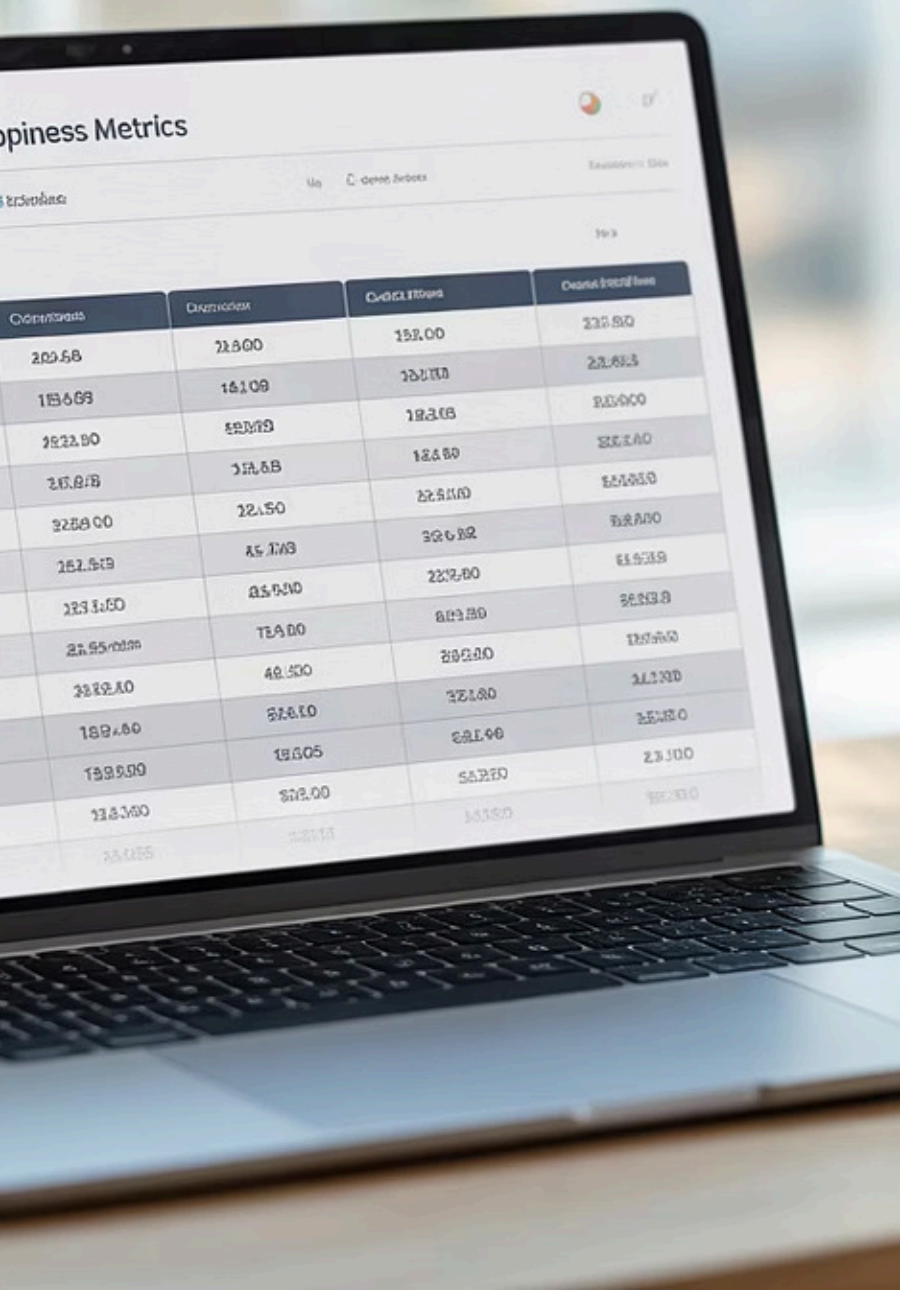
Goals:

1. Investigate whether there is a **trend** indicating that **money makes people happy** using **Better Life Index** and **GDP per capita** data.
2. If the answer for the first question is Yes, select a **life satisfaction model** based on **GDP per capita** to understand the relationship.

Data Sources:

- **Better Life Index** data from the **OECD** website.
- **Gross domestic product (GDP) per capita** from the **World Bank**.





Continents	Population	GDP per capita	Better Life Index
200.58	22.800	152.00	235.80
189.688	16.108	102.113	22.683
1922.80	88.878	183.08	80.400
26.878	15.68	18.68	22.140
3288.00	22.150	22.5110	22.1080
151.549	15.178	30.632	8.8800
223.150	25.940	22.280	61.539
21.950000	17.900	80.380	28.838
22.8210	48.500	28.280	12.7000
189.480	27.610	32.180	21.100
199.800	18.805	28.140	25.880
11.8300	30.200	56.280	2.8300
28.055	22.113	15.180	12.110

Data Integration: Joining the Tables

Process:

- Join the two datasets: **Better Life Index** (happiness data) and **GDP per capita**.
- Sort the resulting data by **GDP per capita**.

Outcome:

You'll have a **combined table** to analyze the relationship between **GDP** and **happiness**.

Example Data Combined in a Dataset

Table 1-1. Does money make people happier?

Country	GDP per capita (USD)	Life satisfaction
Turkey	28,384	5.5
Hungary	31,008	5.6
France	42,026	6.5
United States	60,236	6.9
New Zealand	42,404	7.3
Australia	48,698	7.3
Denmark	55,938	7.6

Plotting the data for the countries

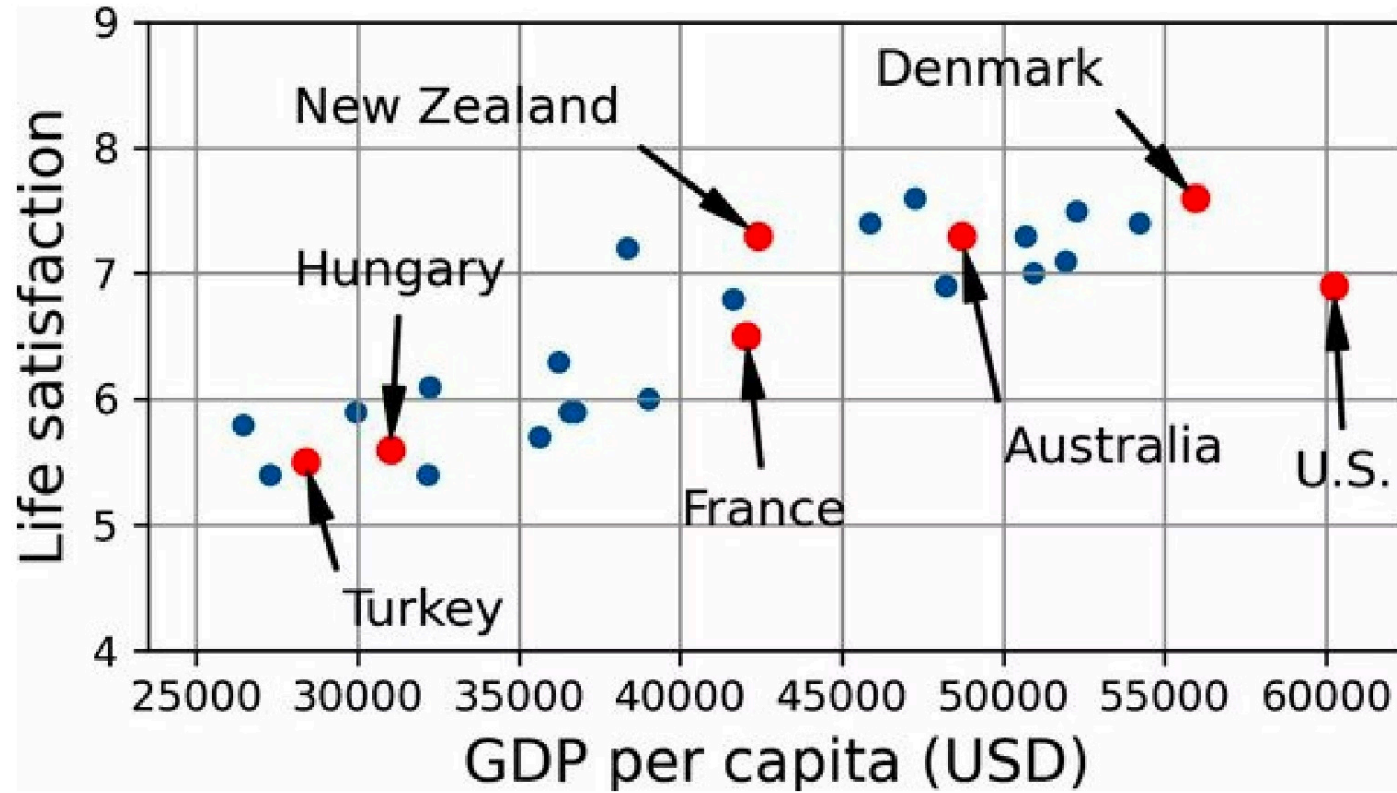


Figure 1-18. Do you see a trend here?

Key Takeaways



Data Integration:

By joining datasets from different sources, we can perform a more comprehensive analysis.

Result of the Analysis Goal:

Based on the first goal of the analysis we identified a trend indicating that **money makes people happy** using **Better Life Index** and **GDP per capita** data.

Model Selection: Analyzing Life Satisfaction

Goals:

1. Investigate whether there is a trend indicating that **money makes people happy** using **Better Life Index** and **GDP per capita** data.
2. If the answer for the first question is Yes, select a **life satisfaction model** based on **GDP per capita** to understand the relationship.

Observation:

- **Life satisfaction** appears to increase **linearly** as **GDP per capita** increases.





Choosing a Model: Linear Regression

- **Decision:**
 - You decide to model **life satisfaction** as a **linear function** of **GDP per capita**.

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

- **Why Linear Model?**
 - The **linear model** is **selected** based on the **observed linear trend** in the **data**.

Some Examples of Linear Models

- This model has two model parameters (θ_0 and θ_1)
 - By tweaking these parameters, you can make your model represent any linear function.

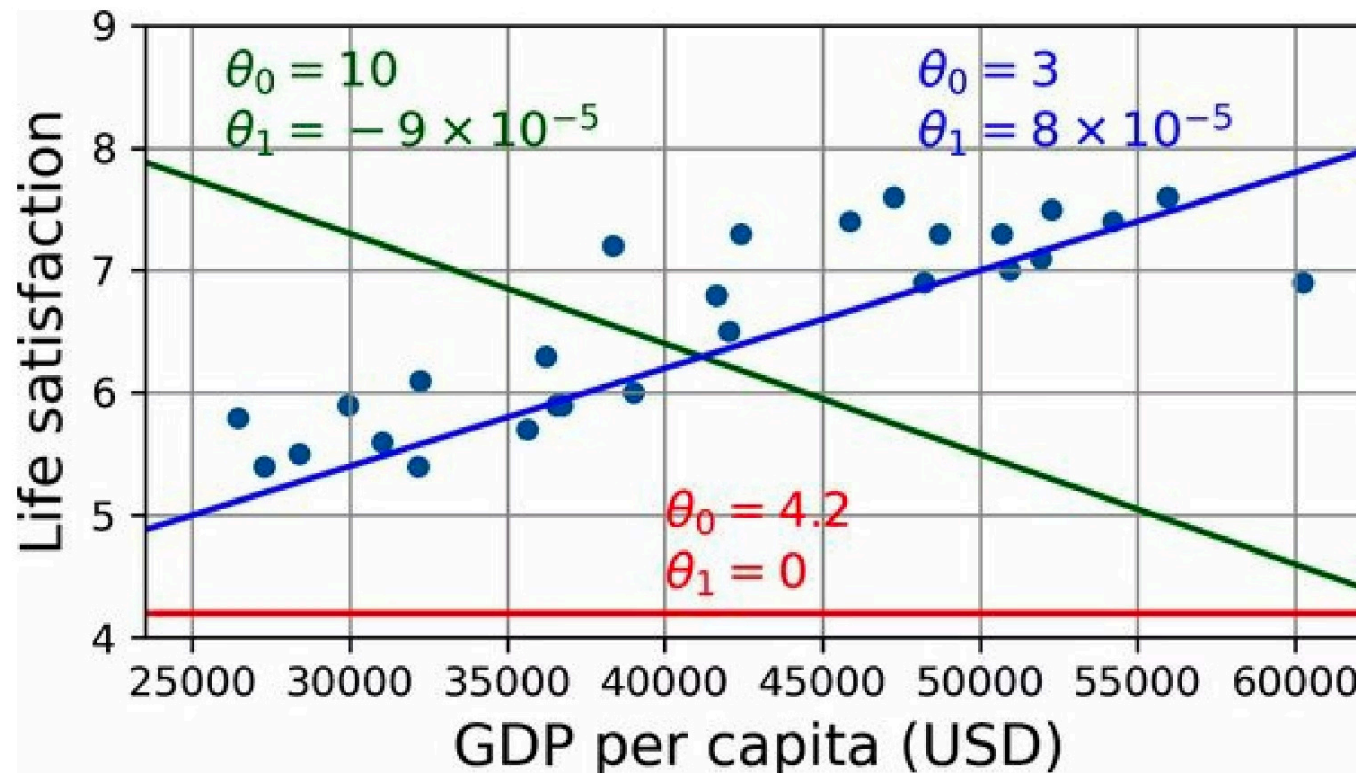


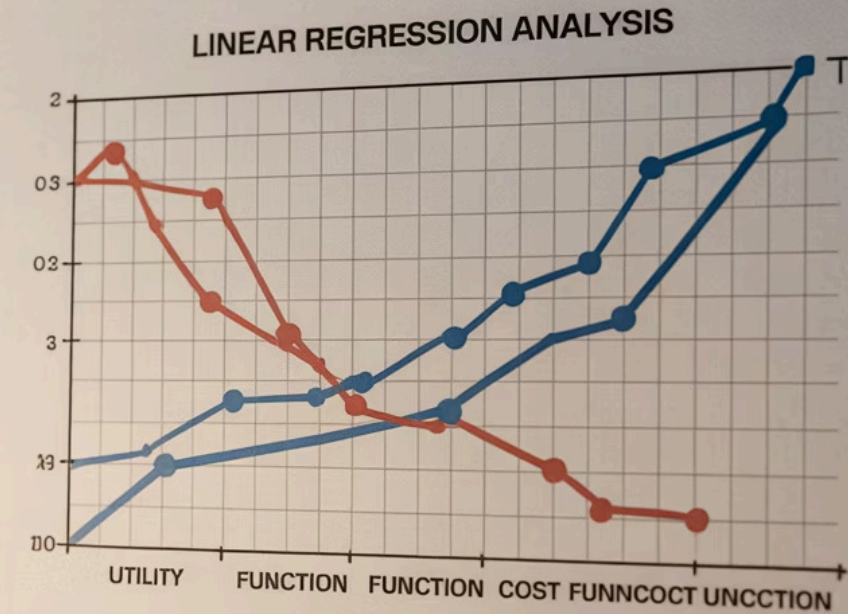
Figure 1-19. A few possible linear models

Defining Model Parameters: θ_0 and θ_1

- Objective:
 - Before using the model, you need to determine the **parameter values** (θ_0 and θ_1) that will make the **model perform best**.
 - How can you know which **values** will make your **model perform best**?
 - Need to specify a **performance measure**.



Performance Measure: Utility vs. Cost Function



- **Utility Function (Fitness Function):**
 - Measures **HOW GOOD** the model is by evaluating its performance.
- **Cost Function:**
 - Measures **HOW BAD** the model is.
 - For linear regression, the cost function typically measures the distance between the model's predictions and the actual training examples.

Training the Model

- What does it mean to train the model?
 - You feed the training examples to the linear regression algorithm.
 - The algorithm adjusts the parameters (θ_0 and θ_1) to minimize the cost function,
 - i.e., minimize the error between predictions and actual values.
- Example:
 - After training, the optimal parameter values are found:
 - $\theta_0 = 3.75$
 - $\theta_1 = 6.78 \times 10^{-5}$



Best Fit Linear Model Parameters for the Training Data

- Now the model fits the training data as closely as possible.

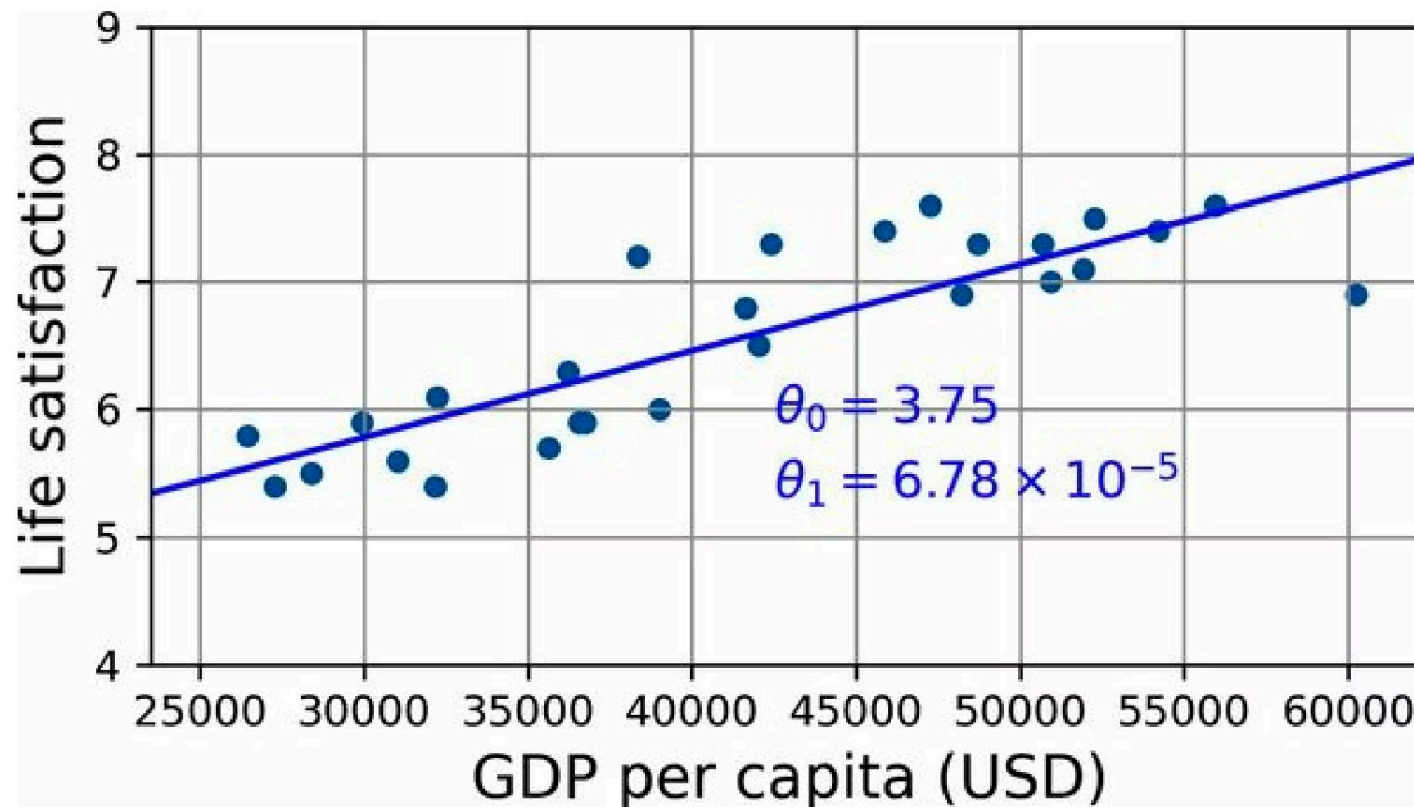
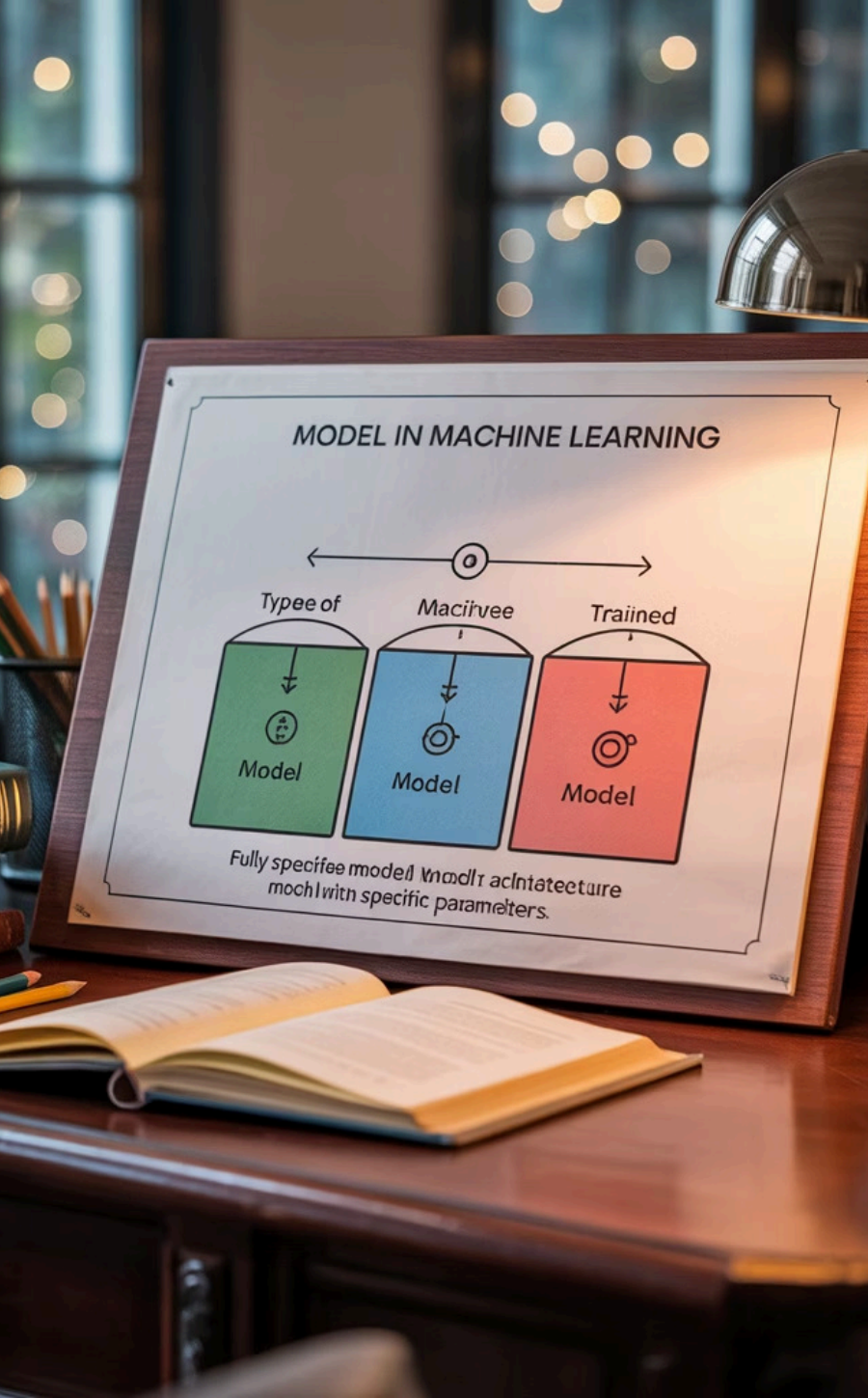


Figure 1-20. The linear model that fits the training data best

Understanding "Model" in Machine Learning

- **Confusing Terminology:**

- The term "model" can refer to different things in ML:
 - **Type of model:** e.g., Linear regression.
 - **Fully specified model architecture:** e.g., Linear regression with one input and one output.
 - **Trained model:** e.g., Linear regression with parameters $\theta_0 = 3.75$ and $\theta_1 = 6.78 \times 10^{-5}$ after training.



Model Selection: Choosing the Right Model

- **Definition:**
 - **Model selection** is the process of:
 - **Choosing the type of model** (e.g., linear regression, decision tree).
 - **Fully specifying its architecture** (e.g., number of inputs, outputs, and parameters).



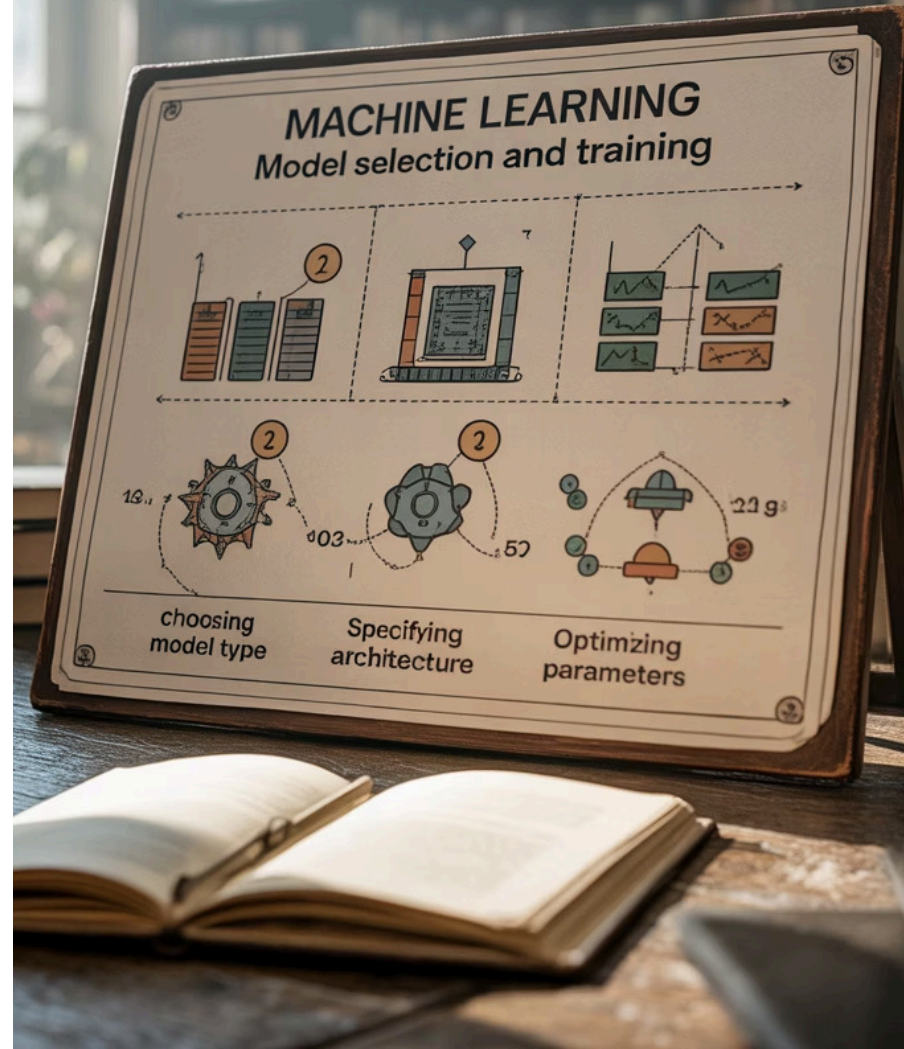


Training a Model

- **Definition:**
 - **Training a model means:**
 - Running an algorithm to **find the model parameters** (θ_0 and θ_1 in linear regression) that **best fit** the training data.
 - The goal is to **minimize error** and **optimize the model** so it can make accurate predictions on new data.

Key Takeaways

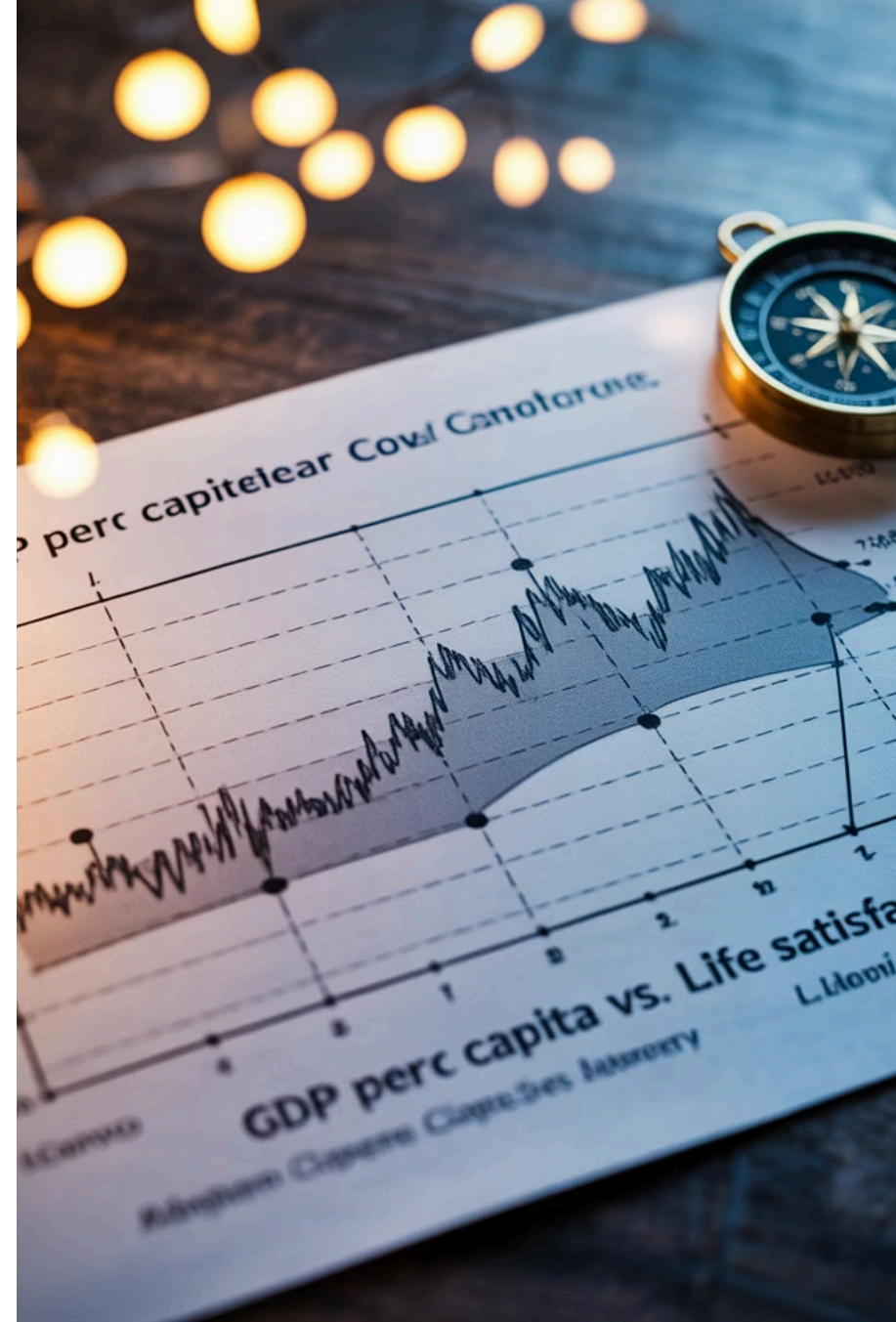
- **Model Selection:**
 - Choose the **type of model** and **specify its architecture** before training.
- **Training:**
 - **Training** is the process of finding the **optimal parameters** to make the model fit the data well.



- **Model Selection:**
 - **Model selection** involves choosing the **best model** based on the **data**.
 - In this case, a **linear regression** model was **chosen for life satisfaction**.
- **Linear Relationship:**
 - **GDP per capita** is used as the single **predictor** for **life satisfaction**.

Key Takeaways

- **Performance Measurement:**
 - Use a **cost function** to evaluate **HOW BAD** the model is and **adjust the parameters** accordingly.
- **Training the Model:**
 - **Training** involves using the **cost function** to **adjust parameters** until the model fits the data well.





Using a Model to Make Predictions

- **Goal:**
 - Predict life satisfaction for Cypriots based on GDP per capita.
- **Known Information:**
 - Cyprus's GDP per capita: \$37,655.
 - **Model equation:**
 - Life satisfaction = $\theta_0 + \theta_1 \times \text{GDP per capita}$
 - With $\theta_0 = 3.75$ and $\theta_1 = 6.78 \times 10^{-5}$.
- **Prediction:**
 - Applying the model:
 - Life satisfaction = $3.75 + 37,655 \times 6.78 \times 10^{-5} = 6.30$.

Key Takeaways

- **Making Predictions:**
 - After training the model, you can use **GDP per capita** to predict **life satisfaction**.
- **Python for ML:**
 - **Python** is a powerful tool for loading data, training models, and making predictions.



Example: Training and running a linear model using Scikit-Learn

- **Steps in Python:**
 - **Load Data:** Load the dataset for analysis.
 - **Separate Inputs and Labels:** Split the data into inputs (X) and labels (y).
 - **Visualize Data:** Create a scatterplot for visualization.
 - **Train the Model:** Train a linear regression model.
 - **Make a Prediction:** Use the model to make predictions based on input data.
- **Code Overview:**
 - The Python code trains the model and predicts **life satisfaction** based on Cyprus's GDP.
- **Scikit-Learn** will be presented in the following chapters.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

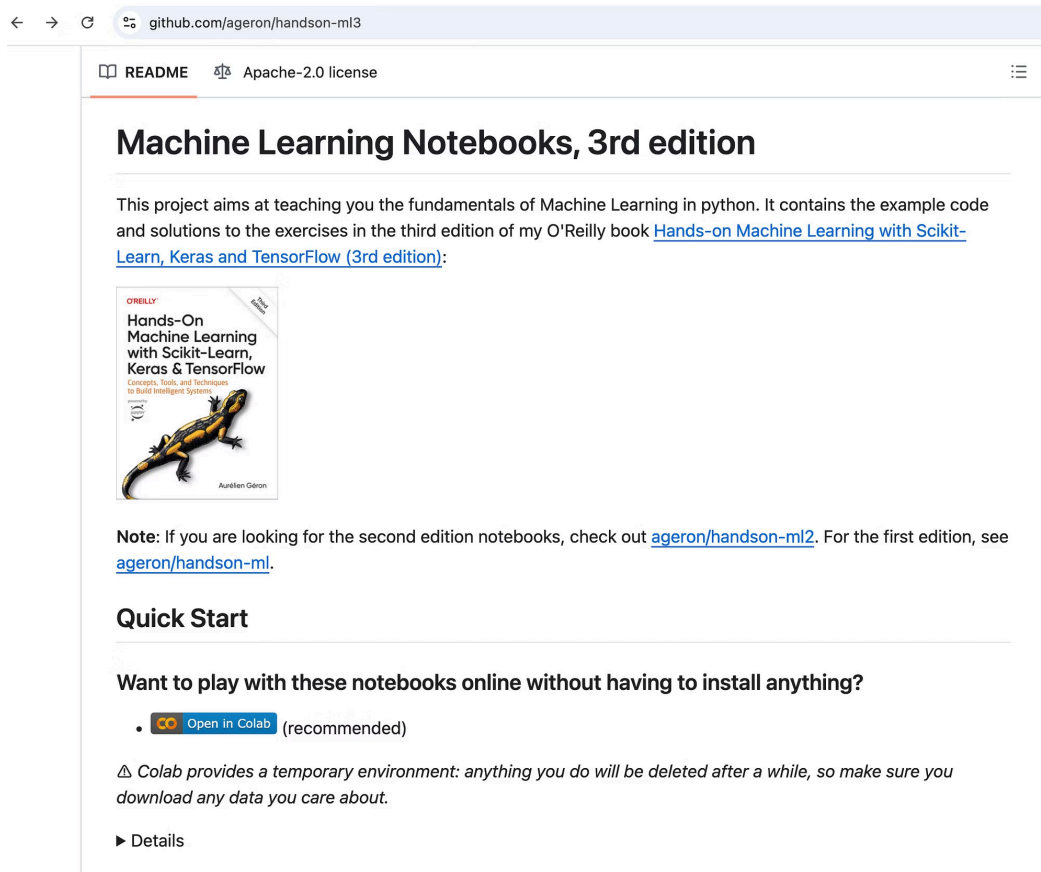
# Download and prepare the data
data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

# Visualize the data
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# Select a linear model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Make a prediction for Cyprus
X_new = [[37_655.2]] # Cyprus' GDP per capita in 2020
print(model.predict(X_new)) # output: [[6.30165767]]
```



Textbook Examples on GitHub

<https://github.com/ageron/handson-ml3>

github.com/ageron/handson-ml3/blob/main/01_the_machine_learning_landscape.ipynb

Files

main

Go to file

- > .github
- > docker
- > images
 - .gitignore
 - 01_the_machine_learning_landsc...
 - 02_end_to_end_machine_learnin...
 - 03_classification.ipynb
 - 04_training_linear_models.ipynb
 - 05_support_vector_machines.ipy...
 - 06_decision_trees.ipynb
 - 07_ensemble_learning_and_rand...
 - 08_dimensionality_reduction.ipynb
 - 09_unsupervised_learning.ipynb
 - 10_neural_nets_with_keras.ipynb
 - 11_training_deep_neural_network...
 - 12_custom_models_and_training_...
 - 13_loading_and_preprocessing_d...
 - 14_deep_computer_vision_with_c...
 - 15_processing_sequences_using...
 - 16_nlp_with_rnn_and_attention.i...

handson-ml3 / 01_the_machine_learning_landscape.ipynb

Preview Code Blame 1745 lines (1745 loc) · 235 KB

Code example 1-1

In [5]:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

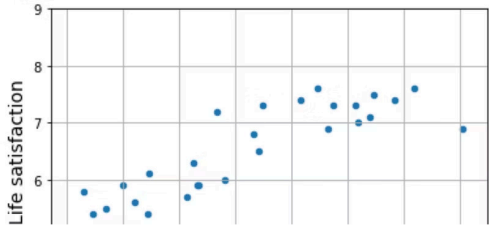
# Download and prepare the data
data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

# Visualize the data
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# Select a linear model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Make a prediction for Cyprus
X_new = [[37_655.2]] # Cyprus' GDP per capita in 2020
print(model.predict(X_new)) # outputs [[6.30165767]]
```



GitHub Repository with Notebook Examples

<https://tinyurl.com/4wam2n3y>

Google Colab with Notebook Examples

<https://tinyurl.com/sa773mxxp>

colab.research.google.com/github/ageron/handson-ml3/blob/main/01_the_machine_learning_landscape.ipynb Save in GitHub to keep changes

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all Copy to Drive

Code example 1-1

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

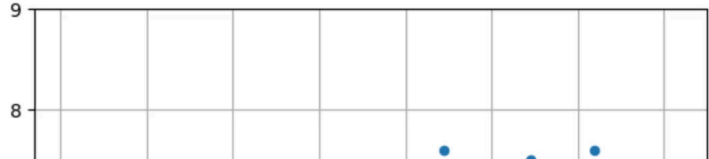
# Download and prepare the data
data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

# Visualize the data
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# Select a linear model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Make a prediction for Cyprus
X_new = [[37_655.2]] # Cyprus' GDP per capita in 2020
print(model.predict(X_new)) # outputs [[6.30165767]]
```



{} Variables [Terminal]

Next: Summary: Types of ML Systems

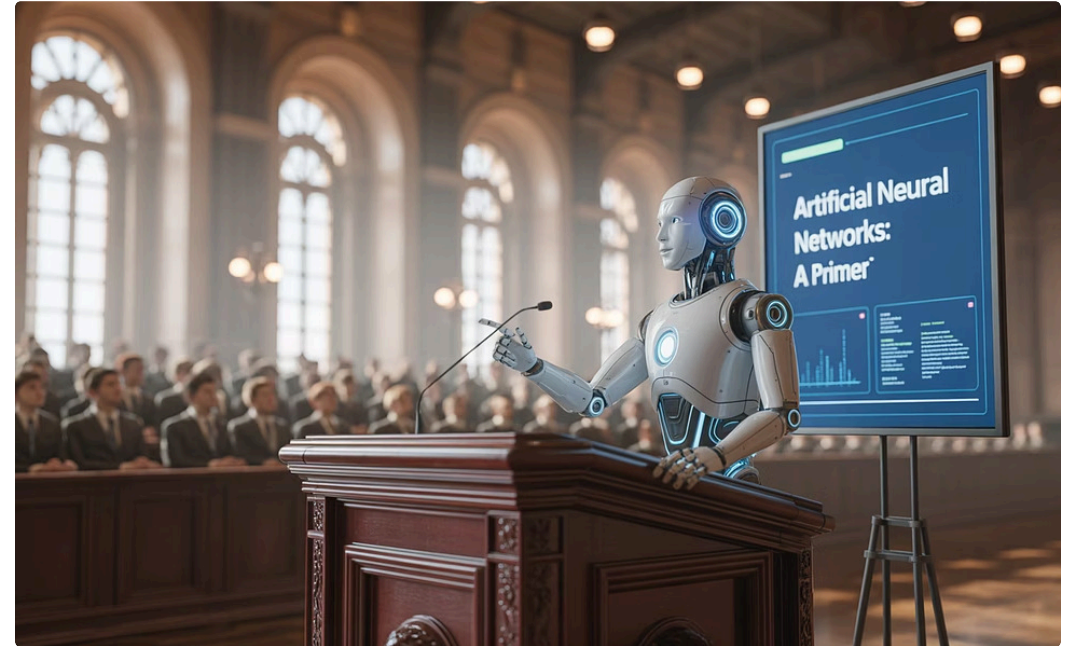


Q&A



Open Floor

Open floor for discussion or clarifications



Discussion

Share your thoughts and questions about AI concepts